

Gaussian Mixture Models

*Date: April 2, 2026**Eric Wong*

Acknowledgements. These notes are based on these [excellent CS 4/5780 notes](#) from Cornell, as well as the [CS229 notes](#) from Stanford, but adjusted to conform to the notation we’ve been using in this course.

1 Soft Clustering

Previously, we looked at the k -means clustering algorithm, which solved an unsupervised learning problem. In this problem, the goal was to partition the data into clusters that had high similarity within each cluster. However, the k -means algorithm has two main downsides:

1. k -means is restricted to assigning each point to only a single cluster (single-membership). This is a “hard” assignment of points to clusters—it seems reasonable to think that points along the boundary between two clusters should be less assigned to each cluster than points that are near the mean of the cluster.
2. k -means with a traditional ℓ_p norm distance metric is restricted to representing spherical clusters. Real clusters may not be so regular. We’ll deal with this later.

For now, let’s first think about how we can relax the hard assignment of points to clusters. Recall that we are given some data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ (e.g., feature vectors only). Our goal in k -means was to partition the data into cluster $\mathcal{C}_1, \dots, \mathcal{C}_k$ where $\mathcal{C}_j \subset \{1, \dots, n\}$ such that each cluster was highly similar.

An equivalent way to view this hard assignment is to consider a label vector for each datapoint. Rather than representing cluster assignment for datapoint x_i with $i \in \mathcal{C}_j \Leftrightarrow i \in j\text{th cluster}$, we can instead simply assign x_i the temporary label j . This label can then be represented as a one-hot vector z_i , i.e.

$$z_{ij} = \begin{cases} 1 & \text{if } i \in \mathcal{C}_j \\ 0 & \text{otherwise} \end{cases}$$

Then, k -means can be interpreted as finding cluster labels z_i for each x_i such that all examples with the same label are highly similar.

A naive way to then relax this is to allow the one-hot label vector to contain continuous, fractional values. This would allow, in some sense, partial membership amongst clusters (also called soft-assignment). For example, $z_i = (0.2, 0.8)$ could indicate that the i th example has 0.2 membership towards cluster 1, and 0.8 membership towards cluster 2. Our goal is now to find a soft assignment of points to clusters that minimizes the loss function.

However, our dissimilarity loss function was only defined previously for hard assignment of points to clusters. It may not be immediately obvious how to minimize the “dissimilarity” of a soft

assignment for a cluster¹. To tackle this problem, we will take an approach that we’ve seen many times before that works well for these softer problems—the probabilistic view! This method is called the Gaussian Mixture model.

What we will now see is a probabilistic model for the clustering problem that can reduce to being equivalent to the k -means algorithm.

2 Gaussian Mixture Model

Recall that in machine learning, we also solve problems of the form

$$\min_{\theta} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i) \equiv \min_{\theta} \ell(\theta)$$

and that for the k -means clustering algorithm, we instead minimized

$$\min_{\mathcal{C}_1, \dots, \mathcal{C}_k} \sum_{j=1}^k \ell(\mathcal{C}_j) \equiv \min_{\mathcal{C}_1, \dots, \mathcal{C}_k} \ell(\mathcal{C}_1, \dots, \mathcal{C}_k)$$

where the loss was the dissimilarity loss within each cluster. Our goal is to relax these clusters from a hard partition to a soft assignment. Let $z_i \in \mathbb{R}^k$ be the soft assignment of the i th data point to the k different clusters. Then, a naive attempt would be to solve a problem of the following form:

$$\min_{z_1, \dots, z_n} \ell(z_1, \dots, z_n)$$

But this brings up the obvious question: what loss function do we use for soft assignments? Coming up with the “right” loss function can be hard and unintuitive in many cases. However, we know that the loss minimization view is equivalent to the probabilistic view in many cases. Therefore, instead of minimizing loss, we will instead assume a generative model, and maximize the likelihood. This is not a new strategy: recall that in logistic regression, we assumed a generative model of $\Pr[y = 1|x] = \sigma(w^{\top}x)$, and maximized the likelihood with respect to w . Since we have no labels in clustering, all we need to do is assume a generative model of $\Pr[x|\theta]$, where θ are parameters of the generative model, and maximize the likelihood with respect to θ . In the logistic regression setting, $\theta = w$ was simple to optimize with gradient descent.

Once we have an assumption of $\Pr[x|\theta]$, we can then take our usual approach of maximum likelihood estimation. Specifically, we take the loss to be the negative log likelihood, and assume the data is independent and identically distributed. Then, we can minimize the loss with respect to all of the parameters θ . In the clustering example, this ends up being:

$$\min_{\theta} -\log \mathcal{L}(\theta | X) = \max_{\theta} \sum_{i=1}^n \log \Pr(x_i | \theta)$$

where we can then plug in our generative model for x and solve the optimization problem.

¹A natural way to approach this would be to use some kind of “weighted” dissimilarity according to the soft assignment.

The specific choice of generative model $\Pr(x|\theta)$ is called a mixture of Gaussians. This model assumes that the points x_i are drawn from a mixture of k normal distributions, each with a mean μ_j and covariance matrix Σ_j , each of which is weighted by π_j . Here, $\theta = (\pi, \mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k)$ are the parameters of the Gaussian mixture model. Furthermore, we must assume that $\sum_j \pi_j = 1$ and $\pi_j \geq 0$ for all $j = 1 \dots k$ for π to be a valid mixture. Then, the complete distribution is

$$\Pr(x|\pi, \mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k) = \sum_j \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)$$

which completes the generative model. To now solve this problem, we need to minimize the negative log likelihood, subject to the constraint that π obeys the simplex constraints and that Σ_j are valid covariance matrices for all $j = 1 \dots k$.

Another way to think of this model is to

1. Pick an integer from 1 to k based on π ,
2. Sample from the corresponding Normal distribution $\mathcal{N}(x|\mu_j, \Sigma_j)$.

The resulting distribution is the probability of observing x under this model.

Soft cluster assignment. We motivated this model by trying to find soft cluster assignments. What are these in the model? We can use the contribution of the j th cluster to the total probability of x_i as the soft assignment of x_i to the j th cluster. Specifically, this is

$$z_{ij} = \frac{\pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)}{\sum_{j'=1}^k \pi_{j'} \mathcal{N}(x_i|\mu_{j'}, \Sigma_{j'})}$$

Therefore, if we knew what the parameters $\theta = (\pi, \mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k)$ were, then it would be very easy to calculate the soft cluster assignments.

Estimating the parameters. How do we learn these parameters (μ_j, Σ_j) ? If we had a hard assignment of data points to clusters, this would be easy: for each cluster j , take all of the points assigned to cluster j and compute the MLE estimates for μ_j and Σ_j given these points. In other words, this would be

$$\mu_j = \frac{1}{|\mathcal{C}_j|} \sum_{i \in \mathcal{C}_j} x_i, \quad \Sigma_j = \frac{1}{|\mathcal{C}_j|} \sum_{i \in \mathcal{C}_j} (x_i - \mu_j)(x_i - \mu_j)^\top$$

However, instead of hard assignments, we instead have soft assignments of points to clusters. Fortunately, the MLE for the soft assignment of points to clusters is very similar: simply weight each point and cluster by its cluster membership z_{ij} and normalize by the total cluster membership across points $\sum_{i=1}^n z_{ij}$:

$$\mu_j = \frac{1}{\sum_{i=1}^n z_{ij}} \sum_{i=1}^n z_{ij} x_i, \quad \Sigma_j = \frac{1}{\sum_{i=1}^n z_{ij}} \sum_{i=1}^n z_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top$$

And lastly, the mixture parameters π can be estimated as $\pi_j = \frac{1}{n} \sum_{i=1}^n z_{ij}$. In total, if we knew the soft cluster assignments z_{ij} , then we can easily calculate the optimal parameters $\theta = (\pi, \mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k)$. You can verify that setting z_i to be a one-hot vector does indeed recover the original MLE estimates.

Expectation-Maximization. In total, these two steps of soft assignment and estimation of the parameters are individually easy if the other is known—the distribution parameters make the soft assignment easy, while the soft assignment makes the parameter estimation easy. These two steps are a more general technique called expectation maximization, and is a common approach for optimizing problems with latent variables. In this case, the latent variables are the *hard* assignments of points to cluster z_{ij} for all i and j , using the same generative model.

- **Expectation** refers to the soft cluster assignment step, because this step estimates the expected value of the latent variables $\Pr[x_i \in \mathcal{C}_j] = \mathbb{E}[z_{ij}]$ under the given the distribution $\Pr[x|\theta]$.
- **Maximization** refers to the parameter estimation step, because this step maximizes the likelihood of the data with respect to the parameters, with respect to the expected values of the latents.

You can think of the EM algorithm as a general way that extends the maximum likelihood approach to handling hidden (unobserved) variables. Finally, this EM algorithm for Gaussian mixture models can be shown to be equivalent to the k -means algorithm: simply restrict the covariance to be a diagonal matrix $\Sigma_j = \sigma^2 I$ and let $\sigma \rightarrow 0$. Notice how this reduction captures the assumption that k -means makes on spherical clusters. The proof for this will be in the homework.

2.1 Expectation-Minimization

The EM algorithm is a broader technique for maximizing the likelihood in the presence of latent variables. To generalize the GMM scenario, suppose we assume a probabilistic model $p(x|\theta)$ and wish to maximize the log likelihood

$$\mathcal{L}(\theta) = \sum_{i=1}^n \log \Pr(x_i|\theta)$$

However, we make the additional assumption that the distribution depends on some latent variables. This is equivalent to the GMM assumption that we first pick a random Gaussian using π . This is our latent variable because we do not observe which Gaussian is picked. Instead, we only observe a sample drawn from this randomly picked Gaussian. Therefore, we can only ever estimate how likely it was that a particular Gaussian generated an observation, but never know for sure which one exactly.

In the general EM algorithm, we will not assume we have observed the latent variable z , but instead assume that we know the form of $\Pr(x, z|\theta)$. Again, making the connection back to the GMM algorithm, this corresponds to knowing that $\Pr(x, z | \theta) = \Pr(x | z, \theta) \Pr(z | \theta)$ where $\Pr(x | z, \theta) = \mathcal{N}(\mu_j, \Sigma_j)$ and $\Pr(z | \theta) = \pi_j$ respectively when z is latent corresponding to the j th cluster.

Then, a principled way to maximize the log likelihood is to marginalize over all possible latent variables z :

$$\max_{\theta} \mathcal{L}(\theta) = \sum_{i=1}^n \log \Pr(x_i|\theta) = \sum_{i=1}^n \log \sum_z \Pr(x, z|\theta)$$

This is a hard problem. In the GMM scenario, with k different possible latents z , this is a non-convex problem. Instead of minimizing this directly, we will instead consider minimizing a lower bound on the likelihood. If we can lift the lower bound on the likelihood high enough, then we know that the true likelihood should also be high.

How does this work? Let us consider just one example ($n = 1$) for simplicity. Let Q be any distribution on z , i.e. $\sum_z Q(z) = 1$ and $Q(z) \geq 0$ for all z . Then, we can write

$$\log \Pr(x|\theta) = \log \sum_z \Pr(x, z|\theta) = \log \sum_z Q(z) \frac{\Pr(x, z|\theta)}{Q(z)} = \log \mathbb{E}_Q \left[\frac{\Pr(x, z|\theta)}{Q(z)} \right]$$

Through Jensen's Inequality, we know that for convex f , $\mathbb{E}[f(x)] \geq f(\mathbb{E}[x])$, and so therefore

$$\log \Pr(x|\theta) \geq \mathbb{E}_Q \left[\log \frac{\Pr(x, z|\theta)}{Q(z)} \right] = \sum_z Q(z) \log \frac{\Pr(x, z|\theta)}{Q(z)} = \text{ELBO}(x, Q, \theta)$$

where ELBO stands for "Evidence-based Lower Bound" on the log likelihood. The EM algorithm can be viewed as two alternating steps on maximizing this lower bound:

1. **Expectation:** Fix θ and maximize the ELBO with respect to Q . As we will see later, this corresponds to setting $Q(z) = \Pr(z|x, \theta)$ for the fixed θ . In the GMM algorithm, this was calculating the soft assignment $z_i = Q(z)$ for a fixed $\theta = (\pi, \mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k)$.
2. **Maximization:** Fix Q and maximize the ELBO with respect to θ . In the GMM algorithm, this was updating $\theta = (\pi, \mu_1, \dots, \mu_k, \Sigma_1, \dots, \Sigma_k)$ for the given soft assignments $z_i = Q(z)$

Up until this point, we have left Q as an arbitrary quantity to maximize. The better Q we can find, the better the lower bound. What is the best Q ? It turns out that $Q(z) = \Pr(z|x, \theta)$ is the best possible lower bound. Why? If this is the case, then

$$\log \Pr(x|\theta) \geq \sum_z Q(z) \log \frac{\Pr(x, z|\theta)}{Q(z)} = \sum_z Q(z) \log \frac{\Pr(x, z|\theta)}{\Pr(z|x, \theta)} = \sum_z Q(z) \log \Pr(x|\theta) = \log \Pr(x|\theta)$$

so $Q(z) = \Pr(z|x, \theta)$ is optimal. Thus, the Q step corresponds to simply setting $Q(z) = \Pr(z|x, \theta)$ for the given θ .